ARTIFICIAL INTELLIGENCE LAB
CIE-374P

Faculty Name : Dr. Sunil Maggu

Name : Sameer Sharma
Roll No. : 02314802723
Semester : 6th
Group : 6FSD-1-A

Maharaja Agrasen Institute of Technology,
PSP Area,
Sector - 22, Rohini, New Delhi - 110085

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

VISION

To attain global excellence through education, innovation, research, and work ethics with the commitment to serve humanity.

MISION

M1. To promote diversification by adopting advancement in science, technology, management, and allied discipline through continuous learning

M2. To foster moral values in students and equip them for developing sustainable solutions to serve both national and global needs in society and industry.

M3. To digitize educational resources and process for enhanced teaching and effective learning.

M4. To cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship.

M5. To encourage faculty-student networking with alumni, industry, institutions, and other stakeholders for collective engagement.

MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering

VISION

"To attain global excellence through education, innovation, research, and work ethics in the field of Computer Science and engineering with the commitment to serve humanity."

MISION

M1 To lead in the advancement of computer science and engineering through globally recognized research and quality education.

M2 To prepare students for full and ethical participation in a diverse society and encourage lifelong learning.

M3 To foster development of problem solving, programming and communication skills as an integral component of the profession.

M4 To cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship in the field of computer science and engineering.

M5 To encourage faculty, student’s networking with alumni, industry, institutions, and other stakeholders for collective engagement.

Rubrics for Lab Assessment:

Rubrics		10 Marks			POs and PSOs Covered	
		0 Marks	1 Marks	2 Marks	PO	PSO
R1	Is able to identify and define the objective of the given problem?	No	Partially	Completely	PO1, PO2	PSO1, PSO2
R2	Is proposed design/procedure/algorithm solves the problem?	No	Partially	Completely	PO1,PO2, PO3	PSO1, PSO2
R3	Has the understanding of the tool/programming language to implement the proposed solution?	No	Partially	Completely	PO1,PO3, PO5	PSO1, PSO2
R4	Are the result(s) verified using sufficient test data to support the conclusions?	No	Partially	Completely	PO2,PO4, PO5	PSO2
R5	Individuality of submission?	No	Partially	Completely	PO8, PO12	PSO1, PSO3

a) Experiments according to the list provided by GGSIPU

Experiment No.	Date	Experiment Name	Marks (0-2)					Total Marks (10)	Signature
			R1	R2	R3	R4	R5		
1.		Study of PROLOG.							
2.		Write simple fact for the statements using PROLOG a) Ram likes mango. b) Seema is a girl. c) Bill likes Cindy. d) Rose is red. e) John owns gold.							
3.		Write predicates, one converts centigrade temperatures to Fahrenheit, the other checksif a temperature is below freezing using PROLOG.							
4.		Write a program to implement Breath First Search Traversal.							
5.		Write a program to implement Water Jug Problem.							
6.		Write a program to remove punctuations from the given string.							
7.		Write a program to sort the sentence in alphabetical order.							

8.		Write a program to implement Hangman game using python.							
9.		Write a program to implement Hangman game.							
10.		Write a program to implement Tic-Tac-Toe game.							
11.		Write a program to remove stop words for a given passage from a text file using NLTK.							
12.		Write a program to implement stemming for a given sentence using NLTK.							
13.		Write a program to POS (part of speech) tagging for the give sentence using NLTK.							
14.		Write a program to implement Lemmatization using NLTK.							
15.		Write a program for Text Classification for the given sentence using NLTK.							

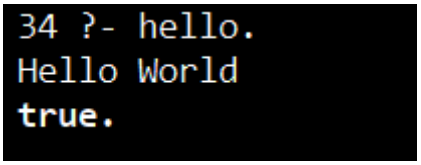
b) Experiments beyond the list provided by GGSIPU

Experiment No.	Date	Experiment Name	Marks (0-2)					Total Marks (10)	Signature
			R1	R2	R3	R4	R5		

SOURCE CODE :

```
hello :-
   write('Hello World').
```

OUTPUT :



EXPERIMENT 2

AIM : Write simple fact for the statements using prolog :

- Ram likes mango.
- Seema is a girl.
- Bill likes Cindy.
- Rose is red.
- John owns gold.

THEORY :

Facts in Prolog are basic assertions that represent true statements about objects and their relationships. A fact consists of a predicate name followed by one or more arguments enclosed in parentheses and ends with a full stop. Facts do not contain conditions and are always considered true if they are stated in the program.

Facts are used to store knowledge in a Prolog database and allow the system to answer queries by pattern matching. They describe simple relationships such as liking, ownership, classification, or properties. By querying facts, Prolog determines whether a given statement matches the stored knowledge, making facts the foundation of knowledge representation in logic programming.

SOURCE CODE :

Facts :-

```
likes(ram, mango).
girl(seema).
likes(bill, cindy).
red(rose).
owns(john, gold).
```

OUTPUT :

```
15 ?- likes(ram, mango).
true.

16 ?-
|    girl(seema).
true.

17 ?- likes(bill, cindy).
true.

18 ?-
|    red(rose).
true.

19 ?-
|    owns(john, X).
X = gold.
```

EXPERIMENT 3

AIM : Write predicates, one converts centigrade temperatures to Fahrenheit, the other checks if a temperature is below freezing using PROLOG.

THEORY :

In Prolog, a predicate represents a logical relationship between one or more arguments and is the fundamental building block of a Prolog program. Predicates are defined using rules and facts, where a rule consists of a head and a body separated by the :- operator. The head specifies the predicate name and parameters, while the body contains conditions that must be satisfied for the predicate to be true.

Predicates allow modular problem solving by dividing a program into logical units that perform specific tasks, such as calculations or condition checking. Prolog predicates can use arithmetic expressions evaluated with the is operator and relational operators like <, >, and = to make decisions. This declarative approach enables Prolog to infer results by matching goals with defined predicates.

SOURCE CODE :

```
% Convert Celsius to Fahrenheit
celsius_to_fahrenheit(C, F) :-
    F is (C * 9 / 5) + 32.

% Check if temperature is below freezing
below_freezing(C) :-
    C < 0.

% Take input from user and display result
check_temperature :-
    write('Enter temperature in Celsius: '),
    read(C),
    celsius_to_fahrenheit(C, F),
    write('Temperature in Fahrenheit: '),
    write(F), nl,
    (   below_freezing(C)
    ->  write('The temperature is below freezing point.')
    ;   write('The temperature is above freezing point.')
    ).
```

OUTPUT :

```
32 ?- check_temperature.
Enter temperature in Celsius: 100.
Temperature in Fahrenheit: 212
The temperature is above freezing point.
true.
```

Experiment – 6

Aim: Write a program to remove punctuations from the given string.

Theory:

In this program, the objective is to remove punctuation marks from a given string using logical predicates. Punctuation marks such as commas, periods, question marks, exclamation marks, and other special symbols are often unnecessary for certain text processing tasks. Therefore, it becomes important to filter out such characters while retaining only meaningful characters like alphabets, digits, and spaces.

The program works by first converting the input string into a list of individual characters using the string_chars/2 predicate. This allows each character to be processed separately. The include/3 predicate is then used to filter the list of characters based on a specific condition. The condition is defined in the helper predicate is_alnum_or_space/1, which checks whether a character is alphanumeric (letters and numbers) or a space using the built-in char_type/2 predicate.

Only characters that satisfy this condition are included in the cleaned list. Finally, the filtered list of characters is converted back into a string using string_chars/2, resulting in a new string without punctuation marks.

Thus, the program demonstrates the use of list processing, character handling, and built-in predicates in Prolog to perform string manipulation efficiently.

Source code:

```
remove_punctuation(Input, Clean) :-string_chars(Input, Chars),
    include(is_alnum_or_space, Chars, CleanChars),
    string_chars(Clean, CleanChars).


is_alnum_or_space(Char) :-
    char_type(Char, alnum);
    char_type(Char, space).
```

Output:

```
?- remove_punctuation("Hello, world! How are you?", Result).
Result = "Hello world How are you".
```

EXPERIMENT-7

Aim: Write a program to sort the sentence in alphabetical order.

Theory:

This program sorts the words of a given sentence in alphabetical order using Prolog predicates. First, the input sentence is separated into individual words using the built-in predicate atomic_list_concat/3, which splits the sentence based on spaces and stores the words in a list.

Next, the list of words is sorted alphabetically using the msort/2 predicate. This predicate arranges the words in ascending order while preserving duplicate words.

Finally, the sorted words are joined back into a single sentence using atomic_list_concat/3 with spaces between them. Thus, the program demonstrates how Prolog can perform text processing through list manipulation, sorting, and string handling.

Source Code:

```
% Main predicate to sort a sentence sort_sentence(Sentence,
SortedSentence) :-
  % 1. Split the sentence into a list of words (atoms)

  % atomic_list_concat handles spaces automatically
  atomic_list_concat(Words, ' ', Sentence),
  % 2. Sort the list alphabetically

  % msort is used instead of sort to preserve duplicate words msort(Words,
  SortedWords),
  % 3. Join the sorted words back into a single atom with spaces atomic_list_concat(SortedWords, ' ',
  SortedSentence).
```

Output:

```
?- sort_sentence('the quick brown fox jumps over the lazy dog', Result).
Result = 'brown dog fox jumps lazy over quick the the'.
```

Experiment - 8

Aim: Write a program to implement Hangman game using python

Theory: Hangman is a word-guessing game that simulates interactive problem-solving. The system randomly selects a word, and the user attempts to guess it letter by letter within a limited number of attempts.

This implementation improves over basic versions by:

- Tracking previously guessed letters
- Preventing invalid input
- Providing visual feedback (hangman stages)
- Maintaining game state properly

Source Code:

```
import random

WORDS = ["python", "game", "hangman", "test", "winning"]

HANGMAN_STAGES = [
    """
     -----
     |   |
         |
         |
         |
         |
    =========
    """,
    """
     -----
     |   |
     O   |
         |
         |
         |
    =========
    """,
    """
     -----
     |   |
     O   |
     |   |
         |
         |
```

```
    ==========
    """,
    """
     -----
     |   |
    O   |
    /|   |
        |
        |
    ==========
    """,
    """
     -----
     |   |
    O   |
    /|\\  |
        |
        |
    ==========
    """,
    """
     -----
     |   |
    O   |
    /|\\  |
    /    |
        |
    ==========
    """,
    """
     -----
     |   |
     O   |
    /|\\  |
    / \\  |
        |
    ==========
    """
]

def choose_word():
    return random.choice(WORDS)

def display_word(word, guessed):
    return " ".join([letter if letter in guessed else "_" for letter in word])
```

```
def play():
    word = choose_word()
    guessed_letters = set()
    attempts = len(HANGMAN_STAGES) - 1

    print(" Welcome to Hangman!")

    while attempts > 0:
        print(HANGMAN_STAGES[len(HANGMAN_STAGES) - 1 - attempts])
        print("Word:", display_word(word, guessed_letters))
        print("Guessed Letters:", " ".join(sorted(guessed_letters)))

        guess = input("Enter a letter: ").lower()

        if not guess.isalpha() or len(guess) != 1:
            print("Invalid input. Enter a single letter.")
            continue

        if guess in guessed_letters:
            print("Already guessed.")
            continue

        guessed_letters.add(guess)

        if guess not in word:
            attempts -= 1
            print("Wrong guess!")

        if all(letter in guessed_letters for letter in word):
            print("\nYou Win! Word was:", word)
            return

    print(HANGMAN_STAGES[-1])
    print("\nYou Lost! Word was:", word)

if __name__ == "__main__":
    play()
```

Output:

```
     -----
     |   |
         |
         |
         |
         |
    =========

Word: _ _ _ _
Guessed Letters:
Enter a letter: g
Wrong guess!
```

```
    -----
    |   |
    O   |
        |
        |
        |
   =========

Word: _ _ _ _
Guessed Letters: g
Enter a letter: t

    -----
    |   |
    O   |
        |
        |
        |
   =========

Word: t _ _ t
Guessed Letters: g t
Enter a letter: e

    -----
    |   |
    O   |
        |
        |
        |
   =========

Word: t e _ t
Guessed Letters: e g t
Enter a letter: s

You Win! Word was: test
```

Experiment - 9

Aim: Write a program to implement Hangman game.

Theory: Hangman is a word-guessing game that simulates interactive problem-solving. The system selects a predefined word, and the user attempts to guess it letter by letter within a limited number of attempts.

This implementation improves over basic versions by:

- Tracking previously guessed letters
- Preventing repeated or invalid input
- Updating the hidden word using list processing
- Maintaining game state through recursion
- Providing clear feedback for correct and incorrect guesses

Source Code:

```
% Start the game
start :-
    Word = [p,r,o,l,o,g],
    length(Word, Len),
    create_hidden(Len, Hidden),
    play(Word, Hidden, 6).

% Create hidden word
create_hidden(0, []).
create_hidden(N, ['_'|T]) :-
    N > 0,
    N1 is N - 1,
    create_hidden(N1, T).

% Game over case
play(_, _, 0) :-
    write('Game Over! You ran out of attempts.'), nl.

% Main game loop
play(Word, Hidden, Attempts) :-
    Attempts > 0,
    write('Word: '), write(Hidden), nl,
    write('Attempts left: '), write(Attempts), nl,
    write('Enter a letter: '),
    read(Guess),
    update_word(Word, Hidden, Guess, NewHidden),
```

```
    (
        Hidden \= NewHidden ->
        write('Correct guess!'), nl,
        NewAttempts = Attempts
        ;
        write('Wrong guess!'), nl,
        NewAttempts is Attempts - 1
    ),
    (
        NewHidden = Word ->
        write('Congratulations! You guessed the word.'), nl
        ;
        play(Word, NewHidden, NewAttempts)
    ).

% Update word
update_word([], [], _, []).
update_word([H|T], [_|T1], Guess, [H|T2]) :-
    H = Guess,
    update_word(T, T1, Guess, T2).
update_word([H|T], [H1|T1], Guess, [H1|T2]) :-
    H \= Guess,
    update_word(T, T1, Guess, T2).
```

Output:

```
?- start.

Word: [_ , _ , _ , _ , _ , _]
Attempts left: 6
Enter a letter: p
Correct guess!

Word: [p , _ , _ , _ , _ , _]
Attempts left: 6
Enter a letter: x
Wrong guess!

Word: [p , _ , _ , _ , _ , _]
Attempts left: 5
Enter a letter: r
Correct guess!

Word: [p , r , _ , _ , _ , _]
```

Attempts left: 5
Enter a letter: o
Correct guess!

Word: [p , r , o , _ , o , _]

Attempts left: 5
Enter a letter: l
Correct guess!

Word: [p , r , o , l , o , _]
Attempts left: 5
Enter a letter: g
Correct guess!

Congratulations! You guessed the word.

Experiment - 10

Aim: Write a program to implement Tic-Tac-Toe game

Theory: Tic-Tac-Toe is a two-player strategy game played on a 3×3 grid where players take turns marking spaces with X and O. The objective is to place three of the same symbols in a row, column, or diagonal to win the game.

This implementation improves over basic versions by:

- Representing the board using lists
- Alternating turns between players
- Validating moves to prevent overwriting positions
- Checking winning conditions using pattern matching
- Maintaining game state using recursion

Source Code:

```
% Start the game
start :-
   Board = [' ',' ',' ',' ',' ',' ',' ',' ',' '],
   play(Board, x).

% Display board
display([A,B,C,D,E,F,G,H,I]) :-
   write(A), write(' | '), write(B), write(' | '), write(C), nl,
   write('---------'), nl,
   write(D), write(' | '), write(E), write(' | '), write(F), nl,
   write('---------'), nl,
   write(G), write(' | '), write(H), write(' | '), write(I), nl.

% Game loop
play(Board, Player) :-
   display(Board),
   (
      win(Board, x) -> write('Player X wins!'), nl, !;
      win(Board, o) -> write('Player O wins!'), nl, !;
      full(Board) -> write('Game Draw!'), nl;
      write('Player '), write(Player), write(', enter position (1-9): '),
      read(Pos),
      ( move(Board, Pos, Player, NewBoard) ->
         switch(Player, Next),
         play(NewBoard, Next)
      ;
```

```
            play(Board, Player)
        )
    ).

% Switch player
switch(x, o).
switch(o, x).

% Valid move
move(Board, Pos, Player, NewBoard) :-
    Pos >= 1, Pos =< 9,
    nth1(Pos, Board, ' '),
    replace(Board, Pos, Player, NewBoard), !.

% Invalid move
move(_, _, _, _) :-
    write('Invalid move! Try again.'), nl,
    fail.

% Replace element
replace([_|T], 1, X, [X|T]).
replace([H|T], Pos, X, [H|R]) :-
    Pos > 1,
    Pos1 is Pos - 1,
    replace(T, Pos1, X, R).

% Win conditions
win([A,A,A,_,_,_,_,_,_], A) :- A \= ' '.
win([_,_,_,A,A,A,_,_,_], A) :- A \= ' '.
win([_,_,_,_,_,_,A,A,A], A) :- A \= ' '.
win([A,_,_,A,_,_,A,_,_], A) :- A \= ' '.
win([_,A,_,_,A,_,_,A,_], A) :- A \= ' '.
win([_,_,A,_,_,A,_,_,A], A) :- A \= ' '.
win([A,_,_,_,A,_,_,_,A], A) :- A \= ' '.
win([_,_,A,_,A,_,A,_,_], A) :- A \= ' '.

% Full board
full([]).
full([H|T]) :- H \= ' ', full(T).
```

Output:

```
?- start.

  |  |
---------
  |  |
---------
  |  |

Player x, enter position (1-9): 1
x |  |
---------
  |  |
---------
  |  |

Player o, enter position (1-9): 5
x |  |
---------
  | o |
---------
  |  |

Player x, enter position (1-9): 2
x | x |
---------
  | o |
---------
  |  |

Player o, enter position (1-9): 8
x | x |
---------
  | o |
---------
  | o |

Player x, enter position (1-9): 3
x | x | x
---------
  | o |
---------
  | o |

Player X wins!
```

Experiment - 11

Aim: Write a program to remove stop words for a given passage from a text file using NLTK

Theory: In Natural Language Processing, textual data often contains many common words such as “is”, “the”, “and”, and “of”, which occur frequently but do not contribute significant meaning to the overall context of the sentence. These words are known as stop words. Removing stop words is an important preprocessing step in text analysis, as it helps in reducing the size of the data and improving computational efficiency.

Source Code:

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

def remove_stopwords(file_path):
    with open(file_path, "r") as f:
        text = f.read()

    words = word_tokenize(text)
    stop_words = set(stopwords.words("english"))

    filtered = [w for w in words if w.lower() not in stop_words]

    print("Original Length:", len(words))
    print("Filtered Length:", len(filtered))
    print("\nFiltered Text:\n", " ".join(filtered))

if __name__ == "__main__":
    remove_stopwords("input.txt")
```

Output:

```
PS C:\Users\Devan\college\aiLab> & c:\python313\python.exe
Original Length: 16
Filtered Length: 8

Filtered Text:
 simple example demonstrate removal stop words sentence .
```

Experiment - 12

Aim: Write a program to implement stemming for a given sentence using NLTK

Theory: Stemming is a text normalization technique used in Natural Language Processing to reduce words to their root or base form by removing suffixes. The primary goal of stemming is to reduce the number of unique words in a dataset, thereby simplifying text analysis and improving the performance of search and retrieval systems.

For example, words such as “playing”, “played”, and “plays” are reduced to a common root form like “play”.

Source Code:

```
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

def stem_text(text):
    ps = PorterStemmer()
    words = word_tokenize(text)

    for w in words:
        print(f"{w} -> {ps.stem(w)}")

if __name__ == "__main__":
    text = input("Enter text: ")
    stem_text(text)
```

Output:

```
PS C:\Users\Devan\college\aiLab> & c:\python313\python.exe
Enter text: playing running studies happily
playing -> play
running -> run
studies -> studi
happily -> happili
```

Experiment - 13

Aim: Write a program to POS (part of speech) tagging for the give sentence using NLTK

Theory: Part-of-Speech (POS) tagging is the process of assigning grammatical categories, such as noun, verb, adjective, and adverb, to each word in a sentence. It is a fundamental task in Natural Language Processing that helps in understanding the syntactic structure and meaning of a sentence.

POS tagging is typically performed using statistical or machine learning models that analyze both the word itself and its context within the sentence

Source Code:

```
import nltk
from nltk.tokenize import word_tokenize

def pos_tagging(text):
    words = word_tokenize(text)
    tags = nltk.pos_tag(words)

    for word, tag in tags:
        print(f"{word}: {tag}")

if __name__ == "__main__":
    text = input("Enter sentence: ")
    pos_tagging(text)
```

Output:

```
PS C:\Users\Devan\college\aiLab> & c:\python313\python.exe c:
Enter sentence: The quick brown fox jumps over the lazy dog
The: DT
quick: JJ
brown: NN
fox: NN
jumps: VBZ
over: IN
the: DT
lazy: JJ
dog: NN
```

Experiment - 14

Aim: Write a program to implement Lemmatization using NLTK

Theory: Lemmatization is the process of converting words into their base or dictionary form, known as the lemma. Unlike stemming, which simply removes suffixes, lemmatization considers the morphological analysis of words and ensures that the resulting word is meaningful and grammatically correct.

For example, the words “running”, “ran”, and “runs” are all reduced to the lemma “run”, while “better” is reduced to “good”

Source Code:

```
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

def lemmatize_text(text):
    lemmatizer = WordNetLemmatizer()
    words = word_tokenize(text)

    for w in words:
        print(f"{w} -> {lemmatizer.lemmatize(w)}")

if __name__ == "__main__":
    text = input("Enter text: ")
    lemmatize_text(text)
```

Output:

```
PS C:\Users\Devan\college\aiLab> & c:\python313\python.exe c
Enter text: running better cars flying
running -> running
better -> better
cars -> car
flying -> flying
```

Experiment - 15

Aim: Write a program for Text Classification for the given sentence using NLTK

Theory: Text classification is a process in Natural Language Processing that involves assigning predefined categories or labels to a given piece of text. It is widely used in applications such as spam detection, sentiment analysis, topic categorization, and email filtering.

The process of text classification typically involves several steps, including data preprocessing, feature extraction, model training, and prediction

Source Code:

```
import nltk
from nltk.classify import NaiveBayesClassifier

def extract_features(text):
    return {word: True for word in text.lower().split()}

def train_model():
    data = [
        ("I love this product", "positive"),
        ("This is amazing", "positive"),
        ("I hate this", "negative"),
        ("This is terrible", "negative"),
        ("Very good experience", "positive"),
        ("Worst service ever", "negative")
    ]

    training = [(extract_features(text), label) for text, label in data]
    return NaiveBayesClassifier.train(training)

def classify_text(classifier, text):
    return classifier.classify(extract_features(text))

if __name__ == "__main__":
    classifier = train_model()

    while True:
        text = input("\nEnter sentence (or 'exit'): ")
        if text.lower() == "exit":
            break
        print("Prediction:", classify_text(classifier, text))
```

Output:

```
PS C:\Users\Devan\college\aiLab> & c:\python313\python.exe

Enter sentence (or 'exit'): I love this product
Prediction: positive

Enter sentence (or 'exit'): This is the worst thing
Prediction: negative
```

Experiment – 5

Aim: Write a program to implement Water Jug Problem.

Theory:

The Water Jug Problem is a puzzle where you are given two empty jugs with different, fixed capacities and an unlimited supply of water.

The goal is to measure a specific amount of water (e.g., exactly 2 liters) into one of the jugs using only three types of actions:

1. Filling a jug to its maximum capacity.
2. Emptying a jug completely.
3. Pouring water from one jug into the other until either the first is empty or the second is full.

The challenge lies in finding the correct sequence of moves to reach the target amount, as you have no markings on the jugs to measure partial amounts directly.

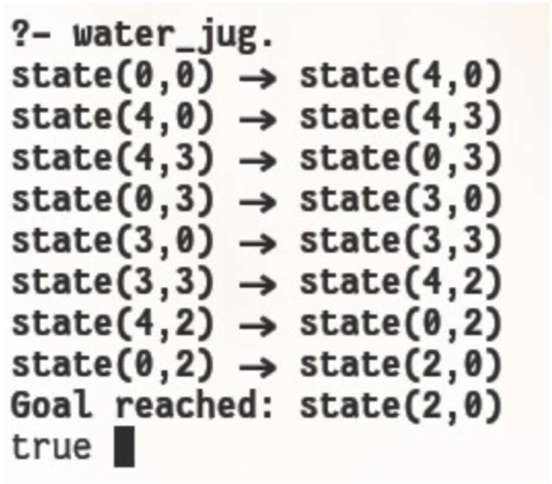
Source Code:

```
% capacity(Jug1, Jug2).
capacity(4, 3).

% goal(Jug1, Jug2).
goal(2, _). % We want 2 liters in the first jug.

% solve(StartJug1, StartJug2)
solve(J1, J2) :-
    path([[J1, J2]], Solution),
    print_solution(Solution).

% Base case: Current state matches the goal
path([[J1, J2]|Rest], [[J1, J2]|Rest]) :-
    goal(J1, J2).

% Recursive case: Try all possible moves
```

```
path([CurrentState|Visited], FullPath) :-
    move(CurrentState, NextState),
    \+ member(NextState, [CurrentState|Visited]), % Avoid cycles
    path([NextState, CurrentState|Visited], FullPath).

% --- Possible Moves ---
% 1. Fill Jug 1
move([J1, J2], [Max1, J2]) :- capacity(Max1, _), J1 < Max1.

% 2. Fill Jug 2
move([J1, J2], [J1, Max2]) :- capacity(_, Max2), J2 < Max2.

% 3. Empty Jug 1
move([J1, J2], [0, J2]) :- J1 > 0.

% 4. Empty Jug 2
move([J1, J2], [J1, 0]) :- J2 > 0.

% 5. Pour Jug 1 into Jug 2 (until Jug 2 is full)
move([J1, J2], [NJ1, NJ2]) :-
    capacity(_, Max2),
    J1 > 0, J2 < Max2,
    Amount is min(J1, Max2 - J2),
    NJ1 is J1 - Amount,
    NJ2 is J2 + Amount.

% 6. Pour Jug 2 into Jug 1 (until Jug 1 is full)
move([J1, J2], [NJ1, NJ2]) :-
    capacity(Max1, _),
    J2 > 0, J1 < Max1,
```

```
    Amount is min(J2, Max1 - J1),
    NJ1 is J1 + Amount,
    NJ2 is J2 - Amount.

% Helper to print the list in the correct order
print_solution([]).
print_solution([State|Rest]) :-
    print_solution(Rest),
    write(State), nl.
```

Output:



Experiment – 4

Aim: Write a program to implement Breath First Search Traversal.

Theory:

Breadth-First Search (BFS) is a fundamental graph traversal algorithm used to explore nodes and edges of a graph systematically. It is particularly known for exploring a graph layer by layer, starting from a chosen root node and visiting all its neighbors at the current depth before moving on to nodes at the next depth level.

Source Code:

```
% Representing the graph: edge(Node, Neighbor)
edge(a, b).
edge(a, c).
edge(b, d).
edge(b, e).
edge(c, f).
edge(e, g).

% bfs(StartNode, ResultingPath)
bfs(Start, Path) :-
    bfs_queue([[Start]], Path).

% bfs_queue(Queue, FinalPath)
% Base Case: The queue is empty.
bfs_queue([], []).

% Recursive Case:
bfs_queue([[Node|PathToNode]|RemainingQueue], [Node|TotalPath]) :-
    % Find all neighbors of the current Node
    findall([Neighbor, Node|PathToNode],
```

```
        (edge(Node, Neighbor), \+ member(Neighbor, [Node|PathToNode])),
        Neighbors),
    % Append neighbors to the BACK of the queue (BFS behavior)
    append(RemainingQueue, Neighbors, NewQueue),
    bfs_queue(NewQueue, TotalPath).
```

Output:

```
?- bfs(a, Path).
Path = [a, b, c, d, e, f, g].

?- bfs(c, Path).
Path = [c, f].

?- bfs(b, Path).
Path = [b, d, e, g].
```